SQLD 제58회 기출 복원문제

각 문항 아래에 정답과 선택지별 풀이를 바로 확인할 수 있습니다.

정답·상세 풀이
총 50문항 · 1과목 10 / 2과목 40

1과목 데이터 모델링의 이해

01 1과목

다음 중 키 엔터티(Key Entity, 기본 엔터티)에 해당하지 않는 것은?

① 사원 **② 주문** ③ 회사 ④ 고객

정답 ②

핵심 해설

키 엔터티(기본 엔터티)는 다른 엔터티의 도움 없이 독립적으로 생성되는 부모 성격의 엔터티이다. 사원·회사·고객은 그 자체로 존재하는 기본 엔터티이지만, 주문은 고객·상품 등 선행 엔터티가 있어야 발생하는 행위 엔터티라 키 엔터티에 해당하지 않는다.

선택지별 풀이

① 독립적으로 생성되는 전형적인 키 엔터티이다.
② 정답. 고객·상품 등 부모 엔터티로부터 발생하는 행위 엔터티이다.
③ 다른 엔터티 없이 존재하는 키 엔터티이다.
④ 외부의 독립적 주체로 키 엔터티에 해당한다.

> **보충 개념** 엔터티는 발생 시점에 따라 키(기본)·메인(중심)·액티브(행위) 엔터티로 구분한다. 키 엔터티는 다른 엔터티의 자식이 되지 않고 스스로 존재하는 엔터티를 말한다.

02 1과목

제3정규형(3NF)에 대한 설명으로 가장 거리가 먼 것은?

① 이행 함수 종속성을 배제하지 않는다.
② 제1정규형의 정의를 만족해야 한다.
③ 제2정규형의 정의를 만족해야 한다.
④ 주식별자를 제외한 일반 속성 간의 함수적 종속성을 제거하는 과정이다.

정답 ①

핵심 해설

제3정규형은 이행적 함수 종속성을 '제거'하는 단계이므로, '배제하지 않는다'는 설명은 정의에 정면으로 어긋난다.

선택지별 풀이

① 정답. 3NF는 이행 종속을 제거하므로 '배제하지 않는다'는 틀린 설명이다.
② 정규화는 누적적이라 3NF는 1NF를 만족해야 한다. 옳다.
③ 3NF는 2NF를 먼저 만족해야 한다. 옳다.
④ 주식별자 외 일반 속성 간 종속(이행 종속) 제거가 3NF의 본질이다. 옳다.

> **보충 개념** 이행적 함수 종속이란 A→B, B→C일 때 A→C가 성립하는 관계이다. 3NF는 이런 간접 종속을 별도 엔터티로 분리해 제거한다.

03 1과목

데이터 모델링을 구성하는 세 가지 관점(View)에 해당하지 않는 것은?

① 데이터 관점 (Data, What)
② 프로세스 관점 (Process, How)
③ 상관 관점 (Interaction)
④ 기술 관점 (Technical)

정답 ④

핵심 해설

데이터 모델링의 세 관점은 데이터(What), 프로세스(How), 그리고 둘의 상관 관점(Interaction)이다. 기술 관점은 여기에 포함되지 않는다.

선택지별 풀이

① 무엇(What)을 다루는 데이터 관점으로 세 관점 중 하나이다.
② 어떻게(How) 처리하는지의 프로세스 관점으로 세 관점 중 하나이다.
③ 데이터와 프로세스의 상호작용을 보는 상관 관점으로 세 관점 중 하나이다.
④ 정답. 기술 관점은 데이터 모델링의 세 관점에 포함되지 않는다.

> **보충 개념** 데이터 모델링의 세 관점은 데이터(정적 구조), 프로세스(동적 처리), 상관(데이터-프로세스 연관)으로 요약된다.

---

04 1과목

데이터 모델링의 유의점 중 '중복성(Duplication)'에 대한 설명으로 가장 적절한 것은?

① 동일한 정보가 여러 엔터티에 중복 저장되어 데이터의 불일치가 발생할 수 있다.
② 비즈니스 프로세스 변화에 따라 모델이 유연하게 대처하지 못하는 상태를 의미한다.
③ 데이터 모델의 독립성을 확보하여 데이터 품질을 높이는 과정이다.
④ 데이터의 타입과 길이를 일관되게 유지하여 무결성을 보장하는 것이다.

정답 ①

핵심 해설

모델링 유의점에서 중복성은 동일 정보가 여러 곳에 저장되어 갱신 시 데이터 불일치(이상현상)를 유발할 수 있다는 점을 경계하는 것이다.

선택지별 풀이

① 정답. 중복 저장으로 인한 데이터 불일치 위험을 정확히 설명한다.
② 유연성(Flexibility) 부족에 대한 설명으로 중복성과는 다른 유의점이다.
③ 데이터 독립성에 대한 설명으로 중복성의 정의가 아니다.
④ 일관성/무결성 확보에 대한 설명으로 중복성과 직접 관련이 없다.

> **보충 개념** 데이터 모델링의 3대 유의점은 중복(Duplication), 비유연성(Inflexibility), 비일관성(Inconsistency)이다. 중복은 한 정보를 여러 곳에 저장해 무결성을 깨뜨릴 위험을 의미한다.

---

05 1과목

다음 마더보드(Motherboard)와 컴퓨터(Computer) 간의 ERD 관계 해석으로 가장 적절한 것은? (단, 관계선에 'O' 표시가 존재함)

ERD

[컴퓨터] ————————O< [마더보드] (마더보드 쪽 관계선 끝에 O 표시)

① 모든 컴퓨터에는 반드시 마더보드가 하나만 존재해야 한다.
② 컴퓨터에 마더보드를 장착할 수도 있고, 장착하지 않을 수도 있다.
③ 마더보드 엔터티는 컴퓨터 엔터티 없이 단독으로 존재할 수 없다.
④ 식별 관계를 통해 두 엔터티의 생명주기를 완벽히 분리한다.

정답 ②

핵심 해설

관계선의 'O'(원) 표시는 선택적 참여(Optional)를 의미합니다. 마더보드 쪽 끝에 'O'가 있으므로 '컴퓨터는 마더보드를 가질 수도, 갖지 않을 수도 있다'로 해석됩니다.

선택지별 풀이

① '반드시 하나만'은 필수 참여(|)의 해석으로, 선택 기호 'O'와 모순됩니다.

② **정답.**

③ 마더보드의 필수 참여를 의미하나, 선택 기호는 컴퓨터→마더보드 방향에 위치합니다.

④ 식별/비식별 관계는 부모의 주식별자를 자식이 상속하는지에 대한 것으로, 생명주기 분리와는 직접 관련이 없습니다.

> **보충 개념** : 관계 표기에서 O(동그라미)=선택적 참여(0 가능), |(막대)=필수 참여(최소 1), 까마귀발(crow's foot)=다(多, N)를 의미합니다.

06 1과목

비식별자 관계(Non-identifying Relationship)에 의해 속성을 생성하는 경우로 가장 적절하지 않은 것은?

① 여러 엔터티가 하나의 엔터티로 통합되었더라도 각 엔터티가 별도의 관계를 가지는 경우

② 각 엔터티의 데이터 생명주기(Life Cycle)를 동일하게 관리하는 경우

③ 자식 엔터티에서 받은 속성(FK)이 반드시 필수가 아니어도 무방한 경우

④ 자식 엔터티에서 주식별자의 사용이 가능하더라도 별도의 주식별자를 생성하는 경우

정답 ②

핵심 해설

부모와 자식의 생명주기를 동일하게 관리하는 경우는 강한 결합인 식별자 관계의 특징이다. 비식별자 관계는 두 엔터티의 생명주기를 별개로 관리할 때 사용하므로, ②는 비식별자 관계의 사례로 가장 적절하지 않다.

선택지별 풀이

① 통합 후에도 별도 관계가 필요해 약하게 연결하는 비식별자 관계의 사례이다.

② **정답. 생명주기를 동일하게 관리하는 것은 식별자 관계의 특징이라 비식별자 관계로 부적절하다.**

③ 부모의 FK가 NULL을 허용(선택적)해도 되는 상황은 비식별자 관계의 전형적 사례이다.

④ 부모 식별자를 받을 수 있어도 별도 주식별자를 두는 것은 비식별자 관계의 사례이다.

> **보충 개념** 식별자 관계는 부모의 PK가 자식의 PK 일부가 되어 생명주기가 강하게 종속되고, 비식별자 관계는 부모 PK가 자식의 일반 속성(FK)으로만 상속되어 생명주기를 독립적으로 관리한다.

07 1과목

관계형 데이터베이스 정규화 과정 중, '이행적 함수 종속성(Transitive Functional Dependency)'을 제거하는 단계는?

① 제1정규화 ② 제2정규화 **③ 제3정규화** ④ BCNF 정규화

정답 ③

핵심 해설

이행적 함수 종속성(A→B→C로 인한 A→C)을 제거하는 단계는 제3정규화(3NF)이다.

선택지별 풀이

① 1정규화는 속성의 원자값(반복그룹 제거)을 보장하는 단계이다.

② 2정규화는 부분 함수 종속을 제거하는 단계이다.

③ **정답. 이행적 함수 종속 제거는 3정규화의 핵심이다.**

④ BCNF는 결정자이면서 후보키가 아닌 함수 종속을 제거하는 단계이다.

> **보충 개념** 정규화 단계별 제거 대상: 1NF=원자값(반복그룹), 2NF=부분 종속, 3NF=이행 종속, BCNF=결정자 이상이다.

08 1과목

두 개 이상의 하위 엔터티 중 하나만 배타적으로 관계를 맺는 '상호배타적 관계(Arc Relationship)'의 사례로 가장 적절한 것은?

① 부서와 사원 ② 고객과 주문 ③ 교수와 학생 **④ 법인차량과 개인차량**

정답 ④

핵심 해설

상호배타적(Arc) 관계는 여러 엔터티 중 어느 하나와만 배타적으로 관계를 맺는 구조이다. 차량이 법인차량이거나 개인차량 중 하나에만 속하는 경우가 이에 해당한다.

선택지별 풀이

① 부서와 사원은 일반적인 1:N 관계로 배타성이 없다.
② 고객과 주문도 단순 1:N 관계이다.
③ 교수와 학생은 별개의 관계로 배타적 선택 구조가 아니다.
④ **정답. 차량이 법인/개인 중 하나에만 속하는 전형적인 배타적(Arc) 관계이다.**

> **보충 개념** Arc(배타적) 관계는 한 엔터티가 둘 이상의 다른 엔터티 중 정확히 하나와만 관계를 맺어야 할 때 사용하며, ERD에서는 호(arc) 기호로 표현한다.

09 1과목

다음 학생과 수강 엔터티 간의 ERD에 대한 설명으로 가장 적절한 것은?

ERD

[학생](학번 PK) 1 ————————O< N [수강](수강번호 PK, 학번 FK) (수강 쪽은 선택 참여)

① 수강번호는 학생별로 중복되어 부여될 수 있다.
② 학생 테이블에 데이터가 존재하면 수강 목록에도 무조건 존재해야 한다.
③ **동일한 수강번호를 여러 학생이 공유하여 가질 수 없다.**
④ 한 명의 학생은 반드시 하나의 과목만 수강해야 한다.

정답 ③

핵심 해설

수강번호가 수강 엔터티의 주식별자(PK)이므로 각 수강번호는 단 하나의 행만 식별합니다. 따라서 동일한 수강번호를 여러 학생이 공유할 수 없습니다.

선택지별 풀이

① 수강번호가 PK이므로 중복 부여될 수 없습니다.
② 수강 쪽 참여가 선택적(O)이므로, 학생이 존재해도 아직 수강하지 않았을 수 있어 수강 목록에 반드시 존재하지는 않습니다.
③ **정답.**
④ 학생 1 : 수강 N 관계이므로 한 학생이 여러 과목을 수강할 수 있습니다.

> **보충 개념** : 학생(1):수강(N) 관계에서 수강은 자신의 주식별자(수강번호)로 각 행이 고유하게 식별되며, 학번을 외래키(FK)로 가져 어느 학생의 수강인지를 나타냅니다.

10 1과목

트랜잭션(Transaction)의 특성 중, "작업이 모두 반영되거나 아니면 전혀 반영되지 않아야 한다(All or Nothing)"는 부분 수정 불가능 원칙을 의미하는 것은?

① **원자성 (Atomicity)**
② 일관성 (Consistency)
③ 고립성 (Isolation)
④ 지속성 (Durability)

정답 ①

핵심 해설

'All or Nothing', 즉 트랜잭션의 작업이 전부 반영되거나 전부 취소되어 부분 처리가 불가능하다는 원칙은 원자성(Atomicity)이다.

선택지별 풀이

① **정답. 전부 반영 또는 전부 취소를 의미하는 원자성이다.**
② 일관성은 트랜잭션 전후로 DB가 일관된 상태를 유지하는 성질이다.
③ 고립성은 동시 실행 트랜잭션이 서로 간섭하지 않는 성질이다.
④ 지속성은 완료된 트랜잭션 결과가 영구히 보존되는 성질이다.

> **보충 개념** 트랜잭션의 ACID 특성은 원자성(Atomicity), 일관성(Consistency), 고립성(Isolation), 지속성(Durability)으로 구성된다.

2과목 SQL 기본 및 활용

11 2과목

다음 중 문자열 처리 함수의 실행 결과가 나머지 셋과 다른 하나는? (Oracle 기준)

① `SELECT TRIM('xxSQLxx', 'x') FROM DUAL;`
② `SELECT RTRIM('SQLxx', 'x') FROM DUAL;`
③ `SELECT REPLACE('xxSQLxx', 'x') FROM DUAL;`
④ `SELECT LTRIM('xxSQL', 'x') FROM DUAL;`

정답 ①

핵심 해설

②③④는 모두 'SQL'을 반환하지만, ①은 문법 오류(ORA-00907)로 결과가 다르다. 오라클 TRIM은 TRIM([LEADING|TRAILING|BOTH] 문자 FROM 문자열) 형태만 허용하며, TRIM('xxSQLxx', 'x')처럼 콤마로 두 인수를 넘기는 문법은 지원하지 않는다.

선택지별 풀이

① **정답. 콤마 구분 2인수 TRIM 문법은 오라클에서 오류(ORA-00907 missing right parenthesis)가 발생하여 값을 반환하지 못한다.**
② RTRIM('SQLxx','x')는 오른쪽 끝의 'x'들을 제거해 'SQL'을 반환한다.
③ REPLACE('xxSQLxx','x')처럼 세 번째 인수를 생략하면 'x'를 빈 문자열로 치환하여 'SQL'을 반환한다.
④ LTRIM('xxSQL','x')는 왼쪽 앞의 'x'들을 제거해 'SQL'을 반환한다.

> **보충 개념** TRIM은 양쪽 한 종류의 문자만 제거하며 FROM 키워드를 쓴다. RTRIM/LTRIM은 두 번째 인수로 제거할 문자 집합을 받고, REPLACE는 세 번째 인수를 생략하면 대상 문자열을 삭제한다.

12 2과목

아래 SQL 문의 실행 결과로 추출되는 행의 개수는? (단, 데이터에는 'a_c', 'abc', 'acc'가 각각 1건씩 존재한다.)

```
SELECT * FROM TABLE_A
WHERE COL LIKE '%a_c%' ESCAPE '\';
```

① 0 ② **1** ③ 4 ④ 6

정답 ②

핵심 해설

ESCAPE '\'를 지정하면 _의 언더바는 와일드카드가 아니라 '_' 문자 그 자체를 의미한다. 따라서 패턴 %a_c%는 'a', 리터럴 '_', 'c'가 순서대로 포함된 문자열만 매칭한다.

선택지별 풀이

① 'a_c'가 정확히 패턴을 만족하므로 0건이 아니다.
② **정답. 'a_c'만 'a'+리터럴'_'+'c' 패턴을 만족하여 1건이 반환된다.**
③ ESCAPE가 없을 때 '_'를 임의 한 글자로 보면 3건 모두 매칭되지만, 여기서는 ESCAPE로 리터럴 처리되어 4건이 될 수 없다.

④ 전체 데이터는 3건뿐이므로 6건은 불가능하다.

> **보충 개념** LIKE에서 '_'는 임의의 한 글자, '%'는 임의의 0글자 이상을 뜻한다. ESCAPE 문자를 와일드카드 앞에 붙이면 해당 와일드카드를 리터럴 문자로 검색할 수 있다.

13 2과목

정규표현식 패턴 ^\w*$에 매칭되지 않는 것은?

① ER-123 ② abcde ③ SQL58 ④ admin123

정답 ①

핵심 해설

\w는 단어 문자 클래스 [A-Za-z0-9_]를 의미하고 ^\w*$는 문자열 전체가 단어 문자로만 이루어져야 매칭된다. 'ER-123'은 하이픈(-)을 포함하므로 매칭되지 않는다.

선택지별 풀이

① **정답. 하이픈 '-'은 \w에 포함되지 않으므로 ^\w*$ 전체 매칭에 실패한다.**
② 'abcde'는 모두 영문자이므로 매칭된다.
③ 'SQL58'은 영문자와 숫자뿐이므로 매칭된다.
④ 'admin123'은 영문자와 숫자뿐이므로 매칭된다.

> **보충 개념** ^는 문자열 시작, $는 끝, *는 0회 이상 반복을 뜻한다. \w는 영문자·숫자·언더바를 포함하지만 하이픈·공백·점 등 특수문자는 포함하지 않는다.

14 2과목

다음 일련의 DCL 명령어가 수행된 후, 테이블 R에 대해 SELECT 권한을 유지하고 있는 유저를 모두 고르시오.

```
DBA : GRANT SELECT, INSERT, DELETE ON R TO U1;
DBA : GRANT SELECT ON R TO U2 WITH GRANT OPTION;
U2 : GRANT SELECT ON R TO U3;
DBA : REVOKE DELETE ON R FROM U1;
```

① DBA, U1 ② DBA
③ DBA, U1, U3 **④ DBA, U1, U2, U3**

정답 ④

핵심 해설

명령을 순서대로 추적하면 DBA(소유자), U1, U2, U3 모두 SELECT 권한을 유지한다.

선택지별 풀이

① U2(WITH GRANT OPTION으로 SELECT 보유)와 U3(U2로부터 SELECT 부여받음)가 빠져 있다.
② U1은 DELETE만 회수되었을 뿐 SELECT는 그대로 유지하므로 DBA만은 아니다.
③ U2가 누락되었다. U2는 SELECT WITH GRANT OPTION을 받았고 회수되지 않았다.
④ **정답. DBA는 객체 소유자이고, U1은 SELECT 유지(DELETE만 회수), U2는 SELECT 보유, U3는 U2로부터 SELECT를 전달받아 모두 SELECT 권한을 가진다.**

> **보충 개념** GRANT는 권한 부여, REVOKE는 회수다. WITH GRANT OPTION을 받은 사용자는 같은 권한을 다른 사용자에게 재부여할 수 있다. 마지막 REVOKE는 U1의 DELETE만 대상으로 하므로 U1의 SELECT/INSERT에는 영향이 없다.

15 2과목

다음 [CUST] 테이블과 [GRADE] 테이블 데이터가 주어졌을 때, 아래 3개의 SQL 문을 실행한 결과값(반환되는 행의 수)의 총합(ㄱ+ㄴ+ㄷ)으로 가장 적절한 것은?

CUST

CUST_ID	AMOUNT
1	100
2	120
3	200

GRADE

GRADE	MIN_AMT	MAX_AMT
A	250	350
B	150	249
C	50	149
D	0	49

```
-- (ㄱ)
SELECT COUNT(*) FROM CUST CROSS JOIN GRADE;
-- (ㄴ)
SELECT COUNT(*) FROM CUST A
INNER JOIN GRADE B ON A.AMOUNT BETWEEN B.MIN_AMT AND B.MAX_AMT;
-- (ㄷ)
SELECT COUNT(*) FROM CUST A
FULL OUTER JOIN GRADE B ON A.AMOUNT BETWEEN B.MIN_AMT AND B.MAX_AMT;
```

① 17 ② 18 ③ 19 **④ 20**

정답 ④

핵심 해설

(ㄱ)12 + (ㄴ)3 + (ㄷ)5 = 20이다. (ㄱ) CROSS JOIN은 카티션 곱이므로 3 × 4 = 12행. (ㄴ) INNER JOIN(BETWEEN): 100→C, 120→C, 200→B로 각각 1건씩 매칭되어 3행. (ㄷ) FULL OUTER JOIN: 매칭 3행 + 미매칭 좌측(CUST) 0행(3건 모두 매칭됨) + 미매칭 우측(GRADE) 2행(A[250~350], D[0~49]는 매칭되는 AMOUNT가 없음) = 5행.

선택지별 풀이

① 17은 (ㄷ)을 3으로 잘못 본 경우의 값으로, 미매칭 GRADE 2건을 더하지 않았다.

② 18은 계산 오류다.

③ 19는 계산 오류다.

④ **정답. 12 + 3 + 5 = 20이다.**

> **보충 개념** FULL OUTER JOIN은 INNER 결과에 더해 양쪽의 미매칭 행을 모두 포함한다. BETWEEN 조인은 등가 조인이 아니므로 범위에 들어가는 행만 결합되고, 어느 행과도 결합되지 못한 GRADE 행은 우측 미매칭으로 별도 출력된다.

16 2과목

온라인 쇼핑몰의 결제 정보를 관리하는 다음 [결제내역] 테이블이 있다. 2025년 12월의 월별 총 결제 건수를 집계하고자 할 때, 다음 중 실행 결과가 올바르게 나오지 않는 SQL 구문은?

결제내역

결제번호	결제일시
PAY001	2025-12-01 10:15:00
PAY002	2025-12-25 18:30:00
PAY003	2026-01-05 09:00:00

① `SELECT COUNT(*) FROM 결제내역 WHERE TO_CHAR(결제일시, 'YYYYMM') = '202512';`

② `SELECT COUNT(*) FROM 결제내역 WHERE 결제일시 >= TO_DATE('20251201', 'YYYYMMDD') AND 결제일시 < TO_DATE('20260101', 'YYYYMMDD');`

③ `SELECT COUNT(*) FROM 결제내역 WHERE SUBSTR(TO_CHAR(결제일시, 'YYYY-MM-DD'), 1, 7) = '2025-12';`

④ `SELECT COUNT(*) FROM 결제내역 WHERE TRUNC(결제일시) = TO_DATE('2025-12', 'YYYY-MM');`

정답 ④

핵심 해설

④는 2025년 12월 전체가 아니라 2025-12-01 하루만 집계하므로 결과가 올바르지 않다. TO_DATE('2025-12','YYYY-MM')은 일(日)이 명시되지 않아 해당 월의 1일(2025-12-01 00:00:00)로 보정된다. TRUNC(결제일시)는 시간만 제거하고 날짜는 보존하므로, 이 비교는 2025-12-01 데이터(PAY001) 1건만 매칭하여 정답 2건과 다르다.

선택지별 풀이

① TO_CHAR로 'YYYYMM'='202512'를 비교해 12월 2건(PAY001, PAY002)을 정확히 집계한다.

② 12월 1일 이상 ~ 다음 해 1월 1일 미만 범위로 12월 2건을 정확히 집계한다.

③ 'YYYY-MM-DD' 문자열에서 앞 7자리('2025-12')를 비교해 12월 2건을 정확히 집계한다.

④ **정답(오답 구문). TO_DATE의 일 보정과 TRUNC 특성 때문에 12월 1일 1건만 집계되어 원하는 결과가 나오지 않는다.**

> **보충 개념** 월 전체를 조건으로 걸 때는 TO_CHAR(컬럼,'YYYYMM') 비교나 >= 1일 AND < 다음달 1일 범위 조건을 사용한다. TO_DATE에서 일을 생략하면 자동으로 1일이 되며, TRUNC(date)는 시·분·초만 잘라내고 일 단위는 유지한다는 점을 구분해야 한다.

---

17 2과목

오라클(Oracle)의 계층형 질의(Hierarchical Query)에 대한 설명으로 가장 틀린 것은?

① `START WITH` 절은 계층 구조의 시작점을 지정하는 역할을 한다.

② `CONNECT BY` 절에 `PRIOR` 키워드를 사용하여 부모와 자식 간의 관계를 정의한다.

③ ORDER SIBLINGS BY 구문은 동일한 부모를 가진 형제 노드 사이의 정렬을 위해 사용된다.

④ `WHERE` 절은 `START WITH` 및 `CONNECT BY` 전개 이전에 먼저 적용되어 계층 구조를 형성할 대상 데이터를 제한한다.

정답 ④

핵심 해설

WHERE 절(조인 조건 제외)은 START WITH와 CONNECT BY로 계층을 전개한 '이후'에 적용되어 결과 행을 걸러낸다. 따라서 '전개 이전에 먼저 적용된다'는 설명은 틀렸다.

선택지별 풀이

① START WITH는 계층 전개의 루트(시작점)를 지정하므로 옳다.

② CONNECT BY의 PRIOR는 부모-자식 연결 방향을 정의하므로 옳다.

③ ORDER SIBLINGS BY는 같은 부모를 가진 형제 노드 간 정렬에 사용되므로 옳다.

④ **정답(틀린 설명). WHERE는 계층 전개 후에 적용되어 개별 행을 필터링하며, 전개 대상 자체를 사전에 제한하지 않는다.**

> **보충 개념** 오라클 계층형 질의의 처리 순서는 (1) START WITH로 루트 선정 → (2) CONNECT BY로 계층 전개 → (3) WHERE로 결과 필터링 순이다. 다만 CONNECT BY 절 내부에 명시한 조인 조건은 전개 과정에서 함께 평가된다.

---

18 2과목

다음 테이블 컬럼에 데이터가 (10, 20, NULL, 30)으로 입력되어 있을 때, AVG(컬럼)의 결과값으로 옳은 것은?

① 15 ② **20** ③ 30 ④ NULL

정답 ②

핵심 해설

AVG는 NULL을 제외하고 계산한다. NULL을 뺀 (10, 20, 30)의 합은 60이고 개수는 3이므로 60 ÷ 3 = 20이다.

선택지별 풀이

① 15는 NULL을 0으로 포함해 60 ÷ 4로 잘못 계산한 값이다.

② **정답. NULL을 제외한 3개 값의 평균은 60 ÷ 3 = 20이다.**
③ 30은 최댓값일 뿐 평균이 아니다.
④ AVG는 일부 값이 NULL이어도 나머지 값으로 평균을 계산하므로 NULL이 아니다.

> **보충 개념** COUNT(*)를 제외한 그룹 함수(AVG, SUM, MAX, MIN, COUNT(컬럼))는 NULL을 무시한다. 평균은 NULL이 아닌 값의 합을 NULL이 아닌 값의 개수로 나눈다.

19 2과목

조인(JOIN) 수행 시 우측 테이블의 모든 데이터를 포함하여 결과를 생성하는 조인 방식으로 가장 적절한 것은? (좌측 테이블은 매칭되는 행만 포함한다)

① RIGHT OUTER JOIN
② LEFT OUTER JOIN
③ FULL OUTER JOIN
④ CROSS JOIN

정답 ①

핵심 해설

RIGHT OUTER JOIN은 우측(오른쪽) 테이블의 모든 행을 결과에 포함하고, 좌측 테이블에 매칭되는 행이 없으면 해당 컬럼을 NULL로 채운다.

선택지별 풀이

① **정답. RIGHT OUTER JOIN은 우측 테이블의 모든 행을 보존한다.**
② LEFT OUTER JOIN은 좌측 테이블의 모든 행을 보존한다.
③ FULL OUTER JOIN은 양쪽 테이블의 모든 행을 보존하므로 '우측만'의 설명과 다르다.
④ CROSS JOIN은 조인 조건 없이 두 테이블의 카티션 곱을 생성한다.

> **보충 개념** OUTER JOIN은 기준이 되는 테이블의 행을 모두 살리는 조인이다. LEFT는 왼쪽, RIGHT는 오른쪽, FULL은 양쪽을 모두 보존하며, 매칭되지 않는 상대 테이블 컬럼은 NULL로 표시된다.

20 2과목

서브쿼리에 대한 설명 중 틀린 것은?

① 다중행 서브쿼리는 단일행 비교 연산자(=)와 함께 쓸 수 있다.
② 서브쿼리는 메인쿼리보다 먼저 실행될 수 있다.
③ 서브쿼리의 반환 건수에 따라 단일행, 다중행으로 나뉜다.
④ 인라인 뷰는 FROM 절에 사용된다.

정답 ①

핵심 해설

다중행 서브쿼리는 여러 행을 반환하므로 단일행 비교 연산자(=, >, < 등)와 함께 쓸 수 없고, IN·ANY·ALL·EXISTS 같은 다중행 연산자를 사용해야 한다. 다중행 결과에 '='를 쓰면 ORA-01427(단일 행 하위 질의에 두 개 이상의 행이 반환됨) 오류가 발생한다.

선택지별 풀이

① **정답(틀린 설명). 다중행 서브쿼리에 '='를 쓰면 오류가 나므로 다중행 연산자를 써야 한다.**
② 서브쿼리(특히 비연관 서브쿼리)는 메인쿼리보다 먼저 실행될 수 있으므로 옳다.
③ 반환 건수에 따라 단일행(1건)·다중행(여러 건) 서브쿼리로 구분되므로 옳다.
④ 인라인 뷰는 FROM 절에 쓰이는 서브쿼리이므로 옳다.

> **보충 개념** 단일행 서브쿼리는 =, >, <, >=, <=, <> 등 단일행 연산자와 쓰고, 다중행 서브쿼리는 IN, ANY/SOME, ALL, EXISTS 등 다중행 연산자와 함께 사용한다. SELECT 절의 스칼라 서브쿼리, FROM 절의 인라인 뷰, WHERE 절의 중첩 서브쿼리로 위치별 분류도 가능하다.

21 2과목

소계 함수(ROLLUP/CUBE)와 함께 사용되는 GROUPING 함수의 특징은?

① 집계 연산으로 생성된 NULL 항목에 대해 1을 반환한다.

② 집계에 사용되지 않은 데이터는 무조건 에러가 난다.

③ 데이터 자체에 들어있는 NULL과 집계로 인한 NULL을 구분할 수 없다.

④ ROLLUP에만 사용할 수 있다.

정답 ①

핵심 해설

GROUPING 함수는 ROLLUP/CUBE/GROUPING SETS 가 만들어 낸 소계·총계 행의 NULL(집계로 인한 NULL)이면 1을, 그렇지 않으면 0을 반환한다.

선택지별 풀이

① 정답.

② 집계에 쓰이지 않은 데이터로 에러가 나지 않는다. GROUPING 은 단지 0/1 을 돌려줄 뿐이다.

③ 정반대다. GROUPING 함수의 존재 이유가 바로 '원본 데이터의 NULL'과 '집계로 생성된 NULL'을 구분하는 것이다.

④ ROLLUP 뿐 아니라 CUBE, GROUPING SETS 에서도 사용할 수 있다.

> **보충 개념** GROUPING(컬럼) 은 해당 행이 그 컬럼에 대해 소계/총계로 집계되어 NULL 이 되었으면 1, 일반 상세 행이면 0 을 반환한다. CASE WHEN GROUPING(col)=1 THEN '합계' ELSE col END 식으로 소계 라벨을 붙이는 데 자주 쓰인다. 여러 컬럼을 묶어 표현하는 GROUPING_ID 함수도 있다.

22 2과목

NTILE(2) 윈도우 함수를 적용했을 때의 결과로 가장 적절한 것은? (단, 데이터는 1, 2, 3, 4 이다.)

① 1, 1, 2, 2 ② 1, 2, 3, 4 ③ 1, 1, 1, 2 ④ 2, 2, 1, 1

정답 ①

핵심 해설

NTILE(2) 는 정렬된 4개 행을 가능한 한 균등하게 2개 그룹으로 나눠 그룹 번호를 매긴다.

선택지별 풀이

① 정답. 4행 ÷ 2그룹 = 그룹당 2행이므로 앞 두 행은 1, 뒤 두 행은 2 → 1, 1, 2, 2.

② ROW_NUMBER() 의 결과이지 NTILE 의 결과가 아니다.

③ 3:1 로 나뉘는 형태로, 균등 분할 원칙과 맞지 않는다.

④ NTILE 그룹 번호는 정렬 순서대로 1부터 증가하므로 2,2,1,1 처럼 역순으로 매겨지지 않는다.

> **보충 개념** NTILE(n) 은 전체 행을 n 개의 버킷으로 나눈다. 행 수가 버킷 수로 나누어떨어지지 않으면 앞쪽 버킷부터 한 행씩 더 채운다 (예: 5행을 NTILE(2) → 3행+2행 = 1,1,1,2,2).

23 2과목

집합 A={1, 2, 3}, B={6, 7}에 대하여 결과 집합이 {(1,6), (2,6), (3,6), (1,7), (2,7), (3,7)}로 출력되었다면, 어떤 조인을 사용한 것인가?

① LEFT OUTER JOIN

② CROSS JOIN

③ INNER JOIN

④ SELF JOIN

정답 ②

핵심 해설

A의 3개 원소와 B의 2개 원소가 모든 조합(3×2=6건)으로 짝지어졌으므로 곱집합(Cartesian Product), 즉 CROSS JOIN 이다.

선택지별 풀이

① LEFT OUTER JOIN 은 조인 조건과 기준 테이블이 필요하며, 모든 조합을 만들지 않는다.

② 정답. 조인 조건 없이 두 집합의 모든 조합을 생성하는 것이 CROSS JOIN 이다.

③ INNER JOIN 은 조인 조건을 만족하는 행만 반환한다. 모든 조합이 나온 것과 다르다.

④ SELF JOIN 은 한 테이블을 자기 자신과 조인하는 것으로, 서로 다른 두 집합 A·B의 조합과 무관하다.

> **보충 개념** CROSS JOIN 의 결과 건수는 두 테이블 행 수의 곱이다. FROM A CROSS JOIN B 또는 조인 조건 없는 FROM A, B 로 표현하며, 결과 컬럼 조합 {(1,6)...(3,7)} 처럼 모든 m×n 짝이 나온다.

24 2과목

다음 조인(JOIN) 문법 중 실행 시 에러가 발생하는 것은?

① `SELECT * FROM T1 NATURAL JOIN T2 ON T1.ID = T2.ID;`

② `SELECT * FROM T1 JOIN T2 USING (ID);`

③ `SELECT * FROM T1 JOIN T2 ON T1.ID = T2.ID;`

④ `SELECT * FROM T1, T2 WHERE T1.ID = T2.ID;`

정답 ①

핵심 해설

NATURAL JOIN 은 동일 이름 컬럼을 자동으로 조인 조건으로 사용하므로 ON 절이나 USING 절을 함께 쓸 수 없다. ON 을 붙이면 구문 오류가 발생한다.

선택지별 풀이

① 정답(에러 발생). NATURAL JOIN 과 ON 을 동시에 사용해 문법 충돌이 난다.

② USING (ID) 는 양쪽의 ID 컬럼으로 조인하는 정상 문법이다.

③ 명시적 ON 조건을 사용하는 표준 내부 조인으로 정상 실행된다.

④ WHERE 절로 조인 조건을 주는 전통적 방식으로 정상 실행된다.

> **보충 개념** NATURAL JOIN, USING, ON 은 조인 조건을 지정하는 세 가지 방식이며 서로 함께 쓸 수 없다. NATURAL JOIN 은 이름이 같은 모든 컬럼을 자동 매칭하고, USING 은 지정한 공통 컬럼만, ON 은 임의의 조건식을 사용한다.

25 2과목

다음 두 테이블이 있다. 아래 SQL문의 실행 결과로 반환되는 값은 무엇인가?

Tbl_A

A	B	C	D
1	a	x	1
2	a	z	1
2	a	y	2
3	b	y	2

Tbl_B

D	E	F
x	D	5
y	E	6
z	E	9

```
SELECT COUNT(*)
FROM (
SELECT DISTINCT A, B
FROM Tbl_A
WHERE C IN (
SELECT D
FROM Tbl_B
WHERE UPPER(E) = 'E'
)
) X;
```

① 1 ② **2** ③ 3 ④ 4

정답 ②

핵심 해설

내부 인라인뷰가 (A,B) 조합 2건을 반환하므로 COUNT(*) 는 2 이다.

선택지별 풀이

① 1건이 아니다. 조건을 만족하는 서로 다른 (A,B) 조합이 2개다.

② **정답. 아래 단계 추적 참조.**

③ 3건이 아니다. DISTINCT 로 (2,a) 의 중복이 1건으로 합쳐진다.

④ 원본 행 수(4)이지, 조건·DISTINCT 적용 후 건수가 아니다.

> **보충 개념** 단계 추적: ①Tbl_B 에서 UPPER(E)='E' 인 행 → (y,E),(z,E) 이므로 D 값 집합 = {y, z}. ②Tbl_A 에서 C IN ('y','z') 인 행 → (2,a,z),(2,a,y),(3,b,y) 3건. ③SELECT DISTINCT A,B → (2,a),(3,b) 2건. ④바깥 COUNT(*) = 2. 서브쿼리는 안쪽부터 평가되며 IN 절은 반환된 값 집합과의 포함 여부를 본다.

26 2과목

COLUMN1의 값이 NULL이 아닌 행을 조회하기 위한 조건절로 올바른 것은?

① `WHERE COLUMN1 IS NOT NULL;` ② `WHERE COLUMN1 <> NULL;`

③ `WHERE COLUMN1 != NULL;` ④ `WHERE COLUMN1 NOT NULL;`

정답 ①

핵심 해설

NULL 여부 비교는 일반 비교 연산자가 아니라 IS NULL / IS NOT NULL 로만 해야 한다.

선택지별 풀이

① **정답. IS NOT NULL 이 NULL 이 아닌 행을 정확히 걸러낸다.**

② <> NULL 은 결과가 항상 UNKNOWN 이 되어 어떤 행도 반환하지 않는다.

③ != NULL 역시 UNKNOWN 으로 평가되어 한 건도 선택되지 않는다.

④ COLUMN1 NOT NULL 은 유효한 WHERE 조건 문법이 아니어서 구문 오류다.

> **보충 개념** SQL 의 3값 논리에서 NULL 과의 산술/비교 연산 결과는 모두 UNKNOWN 이며, WHERE 는 TRUE 인 행만 반환한다. 따라서 NULL 판별은 반드시 IS NULL / IS NOT NULL 을 사용한다.

27 2과목

EXISTS 연산자에 대한 설명으로 가장 적절한 것은?

① **서브쿼리의 결과가 1건이라도 존재하면 TRUE를 반환하고 더 이상 검색하지 않는다.**

② 메인쿼리의 컬럼 값을 서브쿼리로 전달할 수 없다.

③ 결과 집합에 NULL 값이 포함된 경우 무조건 에러를 발생시킨다.

④ 성능상 항상 IN 연산자보다 불리하게 작동한다.

정답 ①

핵심 해설

EXISTS 는 서브쿼리가 행을 한 건이라도 찾는 순간 TRUE 로 확정하고 나머지 탐색을 멈춘다(존재 여부만 판단).

선택지별 풀이

① 정답. 첫 매칭 행 발견 시 즉시 참으로 판정한다.

② EXISTS 는 보통 상관 서브쿼리로, 메인쿼리 컬럼 값을 서브쿼리로 전달해 사용한다.

③ EXISTS 는 행의 '존재 여부'만 보므로 결과에 NULL 이 있어도 에러가 나지 않는다.

④ 성능은 데이터 분포·인덱스에 따라 다르며, 항상 IN 보다 불리한 것은 아니다.

> **보충 개념** EXISTS 는 SELECT 절 내용과 무관하게 '행이 있는가'만 평가하므로 SELECT 1 을 흔히 쓴다. IN 은 서브쿼리 결과 값 집합과의 일치를 보고 NULL 처리에 주의가 필요한 반면, EXISTS 는 존재성만 따져 NULL 의 영향을 덜 받는다.

---

28 2과목

다음 SQL 문에 포함된 정렬 및 추출 옵션의 결과로 올바른 것은?

```
ORDER BY SAL DESC NULLS LAST
FETCH FIRST 3 ROWS ONLY;
```

① 가장 급여가 높은 3명을 출력하되, 급여가 NULL인 데이터는 후순위로 미룬다.

② 가장 급여가 낮은 3명을 출력한다.

③ 가장 급여가 높은 3명을 출력하되, 급여가 NULL인 데이터를 우선 출력한다.

④ 급여와 상관없이 상위 3개의 행만 출력한다.

정답 ①

핵심 해설

SAL DESC 로 급여 내림차순(높은 순) 정렬, NULLS LAST 로 NULL 을 맨 뒤로 보낸 뒤, FETCH FIRST 3 ROWS ONLY 로 상위 3행만 가져온다.

선택지별 풀이

① 정답. 내림차순 + NULL 후순위 + 상위 3건 추출이라는 세 옵션의 의미를 모두 정확히 담았다.

② DESC 는 내림차순이므로 가장 낮은 3명이 아니라 가장 높은 3명이다.

③ NULLS LAST 는 NULL 을 뒤로 미루는 것이라 우선 출력이 아니다.

④ ORDER BY SAL 로 급여 기준 정렬이 적용되므로 '상관없이'가 아니다.

> **보충 개념** FETCH FIRST n ROWS ONLY 는 표준 SQL 의 Top-N 추출 구문으로 Oracle 12c 이상에서 지원한다. Oracle 기본 정렬은 ASC 시 NULLS LAST, DESC 시 NULLS FIRST 이므로, DESC 에서 NULL 을 뒤로 보내려면 NULLS LAST 를 명시해야 한다.

---

29 2과목

RANK() 함수와 DENSE_RANK() 함수의 결과 차이로 올바른 것은? (데이터는 100, 100, 200 이며 오름차순 기준이다.)

① RANK: 1, 1, 3 / DENSE_RANK: 1, 1, 2

② RANK: 1, 2, 3 / DENSE_RANK: 1, 1, 2

③ RANK: 1, 1, 2 / DENSE_RANK: 1, 2, 3

④ 두 함수의 결과는 동일하다.

정답 ①

핵심 해설

오름차순 정렬 시 동점인 100·100 이 공동 1위가 되고, 200 이 그 뒤 순위가 된다. RANK 는 공동 순위 뒤에 건너뛴 순위를 적용하고, DENSE_RANK 는 건너뛰지 않는다.

선택지별 풀이

① 정답. 동점 두 행이 1,1 을 받고, RANK 는 다음 순위가 3(2를 건너뜀), DENSE_RANK 는 2 가 된다.

② 동점인데도 RANK 가 1,2,3 으로 다른 순위를 준 것은 틀렸다(동점은 같은 순위).

③ RANK 결과 1,1,2 는 건너뛰기를 안 한 것이라 RANK 의 동작과 어긋난다.

④ 동점이 있을 때 RANK 와 DENSE_RANK 는 후순위 처리가 달라 결과가 같지 않다.

> **보충 개념** RANK 는 동순위 다음에 '건너뛴' 순위를 부여하여 순위에 공백이 생긴다(1,1,3). DENSE_RANK 는 공백 없이 다음 순위를 연속으로 부여한다(1,1,2). ROW_NUMBER 는 동점이라도 유일한 일련번호(1,2,3)를 매긴다.

30 2과목

다음 [T1] 테이블에 대하여 아래 SQL 문을 실행했을 때, 출력되는 데이터의 순서로 가장 적절한 것은? (Oracle 기준)

T1

COL1	COL2	COL3
A001	2024-12-15	110
A002	NULL	110
B001	2024-12-01	120
C001	2024-12-20	130
D001	2024-12-03	110

```
SELECT COL1, COL2, COL3 FROM T1
ORDER BY COL3 ASC, COL2 ASC;
```

① D001, A001, A002, B001, C001

② A001, A002, D001, B001, C001

③ A002, A001, D001, C001, B001

④ D001, A002, A001, B001, C001

정답 ①

핵심 해설

1차 정렬키 COL3 ASC 로 110 그룹(A001, A002, D001)이 먼저 오고, 그 안에서 2차 정렬키 COL2 ASC 로 12-03(D001) → 12-15(A001) → NULL(A002) 순으로 정렬된다(Oracle ASC 는 NULL 을 마지막에 둔다). 이어서 COL3 가 120 인 B001, 130 인 C001 이 온다.

선택지별 풀이

① 정답. COL3=110 그룹 내에서 COL2 오름차순(12-03, 12-15, NULL 후순위) → D001, A001, A002 순, 이어서 COL3 가 큰 B001, C001.

② COL3=110 그룹 안에서 COL2 가 가장 이른 12-03(D001)이 맨 앞에 와야 하는데 그룹의 맨 뒤에 놓여 COL2 ASC 순서와 어긋난다.

③ NULL 인 A002 를 맨 앞에 두어 Oracle ASC 의 NULL 후순위 원칙과 어긋나고, 110 그룹 안에서도 12-15(A001) 가 12-03(D001) 보다 앞서며, COL3 이 120 인 B001 과 130 인 C001 의 순서까지 뒤바뀌었다.

④ 같은 110 그룹 내에서 NULL(A002) 이 12-03(D001) 다음에 와 12-15(A001) 보다 앞서므로 COL2 ASC 순서와 어긋난다.

> **보충 개념** ORDER BY 는 앞쪽 키부터 우선 적용하고, 값이 같은 그룹 내에서만 다음 키로 정렬한다. Oracle 은 ASC 정렬 시 NULL 을 가장 큰 값처럼 취급해 맨 뒤(NULLS LAST), DESC 시 맨 앞(NULLS FIRST)에 둔다.

31 2과목

다음 중 스칼라 서브쿼리(Scalar Subquery)에 대한 설명으로 올바르지 않은 것은?

① 원칙적으로 단 1개의 행과 1개의 컬럼만 반환해야 한다.

② 메인쿼리의 결과 건수만큼 반복 수행된다.

③ 서브쿼리의 실행 결과가 0건일 경우 에러를 발생시킨다.

④ SELECT 절 등 단일 값이 요구되는 곳에 주로 사용된다.

정답 ③

핵심 해설

스칼라 서브쿼리는 결과가 0건이면 에러가 아니라 NULL을 반환한다. 따라서 ③은 틀린 설명이다.

선택지별 풀이

① 스칼라 서브쿼리는 단일 값(1행 1컬럼)을 반환하는 것이 원칙이다. 2건 이상이면 "single-row subquery returns more than one row" 에러가 난다. 옳은 설명.

② SELECT 절의 스칼라 서브쿼리는 메인쿼리가 반환하는 각 행마다 반복 평가된다. 옳은 설명.

③ 정답. 0건일 때는 에러가 아니라 NULL을 반환한다.

④ SELECT 절, 일부 WHERE 비교 등 단일 값이 필요한 위치에서 사용된다. 옳은 설명.

> **보충 개념** 스칼라 서브쿼리는 하나의 값을 돌려주는 서브쿼리로, 2건 이상이면 에러, 0건이면 NULL이다. SELECT 절에 두면 메인쿼리 행마다 실행되므로 동일 입력에 캐싱(필터 후 캐시)으로 성능을 보완한다.

---

32 2과목

다음 SQL 문의 실행 결과로 가장 적절한 것은?

```
SELECT COUNT(*) FROM DUAL
HAVING COUNT(*) > 4;
```

① 0
② 1
③ 9999
④ 공집합 (No rows selected)

정답 ④

핵심 해설

DUAL은 행이 1건이므로 COUNT(*)는 1이고, HAVING COUNT(*) > 4 즉 1 > 4 가 거짓이 되어 그룹이 걸러져 한 행도 반환되지 않는다.

선택지별 풀이

① 0이 출력되려면 행이 반환되되 카운트가 0이어야 하는데, HAVING이 거짓이면 행 자체가 사라진다. 0행 반환이지 값 0이 아니다.

② COUNT(*)=1이지만 HAVING 조건에서 탈락하므로 1은 출력되지 않는다.

③ 근거 없는 값.

④ 정답. HAVING 조건(1 > 4)이 거짓이라 그룹 전체가 제거되어 공집합이다.

> **보충 개념** GROUP BY가 없어도 집계 함수가 있으면 전체가 하나의 그룹이 된다. HAVING은 이 그룹 단위로 필터링하므로, 조건이 거짓이면 "값 0"이 아니라 "행 없음(No rows)"이 된다. 행이 1건도 없는 테이블에 대해 COUNT(*)만 쓰면 0이 나오지만, HAVING으로 그룹이 탈락하면 결과 자체가 비는 점이 다르다.

---

33 2과목

다음 아래와 같이 테이블을 생성하였다. 다음 중 INSERT 문의 실행 결과가 나머지 셋과 다른 하나는?

```
CREATE TABLE T1 (
COL1 NUMBER,
COL2 NUMBER,
CONSTRAINT CK_T1 CHECK (COL1 > 10 AND COL2 > 5)
);
```

① `INSERT INTO T1 VALUES (20, 3);`
② `INSERT INTO T1 VALUES (5, 3);`
③ `INSERT INTO T1 VALUES (20, NULL);`
④ `INSERT INTO T1 VALUES (5, NULL);`

정답 ③

핵심 해설

CHECK 제약은 결과가 명시적으로 FALSE일 때만 거부하고, UNKNOWN(NULL 비교)이면 허용한다. ③만 입력에 성공하고 나머지는 실패한다.

선택지별 풀이

① (20, 3) → 20>10 TRUE, 3>5 FALSE → TRUE AND FALSE = FALSE → 제약 위반, 실패.

② (5, 3) → 5>10 FALSE → FALSE → 실패.

③ 정답(다른 하나). (20, NULL) → 20>10 TRUE, NULL>5 UNKNOWN → TRUE AND UNKNOWN = UNKNOWN → CHECK는 UNKNOWN을 위반으로 보지 않아 입력 성공.

④ (5, NULL) → 5>10 FALSE → FALSE AND UNKNOWN = FALSE → 실패.

> **보충 개념** CHECK 제약은 3치 논리(TRUE/FALSE/UNKNOWN)에서 FALSE인 경우만 차단한다. NULL이 끼어 UNKNOWN이 되면 통과시키는 점이 핵심이며, AND 연산에서 FALSE는 다른 피연산자가 UNKNOWN이어도 FALSE를 유지한다는 점(④)과 구분해야 한다.

34 2과목

> `GROUP BY ROLLUP(ID, TYPE)` 소계 쿼리에서 `HAVING GROUPING(ID) + GROUPING(TYPE) != 1` 의 결과는?

① 중간 소계 행을 제외한 일반 상세 데이터와 전체 총합계 행만 출력한다.

② 모든 데이터를 출력한다.

③ 오직 전체 총합계 행 한 건만 출력한다.

④ ID별 소계 행만 출력한다.

정답 ①

핵심 해설

GROUPING 함수는 해당 컬럼이 집계로 묶여 NULL이 된 경우 1, 실제 값으로 그룹된 경우 0을 반환한다. ROLLUP(ID, TYPE) 기준으로 두 GROUPING의 합을 보면, 합이 1인 "ID별 중간 소계"만 제외된다.

선택지별 풀이

① 정답. 상세 행은 GROUPING(ID)=0, GROUPING(TYPE)=0 → 합 0(≠1, 출력). ID별 소계 행은 GROUPING(ID)=0, GROUPING(TYPE)=1 → 합 1(=1, 제외). 전체 총합계 행은 GROUPING(ID)=1, GROUPING(TYPE)=1 → 합 2(≠1, 출력). 결국 중간 소계만 빠지고 상세+총합계가 남는다.

② 합이 1인 ID별 소계 행이 제거되므로 모든 데이터는 아니다.

③ 총합계만 남기려면 합이 2인 행만 골라야 한다(!= 1로는 상세 행도 살아남음).

④ ID별 소계는 합이 1이라 오히려 제거되는 대상이다.

> **보충 개념** GROUPING(컬럼)은 그 컬럼이 소계/총계로 묶여 NULL이 된 자리에서 1을 돌려주어, 데이터 본래의 NULL과 집계로 생긴 NULL을 구분한다. ROLLUP(A,B)에서 GROUPING 합은 상세 0, A별 소계 1, 총계 2로 단계가 구분되므로 합 값으로 특정 레벨만 골라낼 수 있다.

35 2과목

GROUPING SETS 함수에 대한 설명으로 올바르지 않은 것은?

① 다양한 소계 집합을 한 번의 쿼리로 효율적으로 구할 수 있다.

② 괄호 내에 명시된 인수의 순서가 달라지면 집계의 결과 항목 자체도 달라진다.

③ 정렬하려면 별도의 ORDER BY 절을 사용해야 한다.

④ UNION ALL을 결합한 것과 동일한 결과를 생성할 수 있다.

정답 ②

핵심 해설

GROUPING SETS는 나열된 각 집합을 독립적으로 집계하므로, 인수(집합)의 나열 순서가 바뀌어도 만들어지는 집계 항목 자체는 동일하다. 정렬 순서만 영향을 받을 수 있을 뿐이라 ②는 틀린 설명이다.

선택지별 풀이

① 여러 GROUP BY 결과를 한 번의 스캔/쿼리로 묶어 구할 수 있다. 옳은 설명.

② **정답(틀린 설명). 순서가 달라져도 집계 결과 집합의 구성은 같다. ROLLUP(A,B)≠ROLLUP(B,A)처럼 순서가 의미 있는 것과 혼동한 것이다.**

③ GROUPING SETS는 정렬을 보장하지 않으므로 원하는 순서가 있으면 ORDER BY를 별도로 써야 한다. 옳은 설명.

④ 각 그룹 집합에 대한 GROUP BY 결과를 UNION ALL로 이어 붙인 것과 동일한 결과를 만든다. 옳은 설명.

> **보충 개념** GROUPING SETS((A),(B),())는 A별, B별, 전체 집계를 각각 만들어 합친 것과 같다. ROLLUP은 컬럼 순서가 계층(소계 단계)을 결정해 순서에 민감하지만, GROUPING SETS는 나열된 집합 자체가 결과라 순서에 무관하다.

36 2과목

NVL2(COL1, 'A', 'B') 함수에서 COL1이 NULL일 경우 반환되는 결과는?

① A ② **B** ③ NULL ④ 에러

정답 ②

핵심 해설

NVL2(expr, val1, val2)는 expr이 NULL이 아니면 val1을, NULL이면 val2를 반환한다. COL1이 NULL이므로 두 번째 대체값 'B'가 반환된다.

선택지별 풀이

① 'A'는 COL1이 NULL이 아닐 때 반환되는 값이다.

② **정답. COL1이 NULL이므로 세 번째 인자 'B'가 반환된다.**

③ NVL2는 인자에 따라 항상 둘 중 하나를 반환하며, NULL을 그대로 돌려주지 않는다.

④ 문법상 정상 함수 호출이라 에러가 아니다.

> **보충 개념** NVL(expr, val)은 NULL이면 val로 치환하는 2인자 함수이고, NVL2(expr, val1, val2)는 NULL 여부에 따라 분기하는 3인자 함수다. "NULL이 아닐 때 val1, NULL일 때 val2" 순서를 헷갈리지 않는 것이 핵심이다.

37 2과목

다음 중 데이터 제어어(DCL)에 해당하지 않고, 데이터 조작어(DML)에 해당하는 것은?

① GRANT ② REVOKE ③ UPDATE ④ COMMIT

정답 ③

핵심 해설

UPDATE는 테이블의 데이터 값을 변경하는 데이터 조작어(DML)이다. 나머지는 권한 제어(DCL) 또는 트랜잭션 제어(TCL)에 해당한다.

선택지별 풀이

① GRANT는 권한을 부여하는 데이터 제어어(DCL)이다.

② REVOKE는 권한을 회수하는 데이터 제어어(DCL)이다.

③ **정답. UPDATE는 행의 값을 수정하는 DML이다.**

④ COMMIT은 트랜잭션을 확정하는 명령으로 보통 TCL로 분류하며, DML은 아니다.

> **보충 개념** DML은 SELECT/INSERT/UPDATE/DELETE처럼 데이터 자체를 다루고, DCL은 GRANT/REVOKE로 권한을 제어한다. COMMIT/ROLLBACK/SAVEPOINT는 트랜잭션을 제어하는 TCL로 별도 분류하는 것이 일반적이다.

38 2과목

```
WHERE SAL > ANY (1000, 2000) 조건절의 논리적 의미와 동일한 것은?
```

① SAL > 1000 OR SAL > 2000 ② SAL > 1000 AND SAL > 2000

③ SAL = 1000 OR SAL = 2000 ④ 에러 발생

정답 ①

핵심 해설

> ANY(목록)은 목록의 값들 중 "어느 하나라도" 만족하면 참이므로 각 비교를 OR로 묶은 것과 같다. 즉 SAL이 목록의 최솟값보다 크면 참이며, SAL > 1000 OR SAL > 2000과 동치다.

선택지별 풀이

① 정답. > ANY는 OR 결합이며, 실질적으로 SAL > MIN(1000) 과 같다.

② AND 결합은 > ALL의 의미다(모든 값보다 커야 함, 즉 SAL > 2000).

③ 등호 OR 결합은 = ANY 즉 IN (1000, 2000)의 의미다.

④ 다중행/다중값 비교에 ANY는 정상 문법이라 에러가 아니다.

> **보충 개념** ANY는 "최솟값보다 크면" 참(OR), > ALL은 "최댓값보다 크면" 참(AND)이다. = ANY는 IN과 같고, != ALL은 NOT IN과 같다는 대응 관계를 함께 기억하면 좋다.

39 2과목

계층형 질의문 SELECT COUNT(*) FROM DUAL CONNECT BY LEVEL <= 2; 의 실행 결과로 옳은 것은?

```
SELECT COUNT(*) FROM DUAL CONNECT BY LEVEL <= 2;
```

① 0 ② 1 **③ 2** ④ 3

정답 ③

핵심 해설

CONNECT BY LEVEL <= 2 는 LEVEL이 1, 2가 될 때까지 행을 전개하여 2개의 행을 생성한다. 그 위에 COUNT(*)를 적용하면 2가 반환된다.

선택지별 풀이

① 행이 생성되므로 0이 아니다.

② DUAL은 본래 1행이지만 CONNECT BY LEVEL <= 2로 행이 2개로 늘어난다.

③ 정답. LEVEL=1, LEVEL=2 두 행이 만들어져 COUNT(*)=2.

④ LEVEL <= 2 조건은 LEVEL=3에서 전개를 멈추므로 3행이 아니다.

> **보충 개념** CONNECT BY LEVEL <= N 패턴은 START WITH 없이도 1행짜리 DUAL을 N행으로 늘리는 대표적인 행 생성(숫자 시퀀스) 기법이다. LEVEL은 1부터 시작해 조건을 만족하는 동안 1씩 증가한다.

40 2과목

데이터 그룹화 조건에서 집계 함수(SUM, COUNT 등)를 활용한 필터링이 가능한 절은?

① WHERE 절 ② GROUP BY 절 **③ HAVING 절** ④ ORDER BY 절

정답 ③

핵심 해설

그룹으로 묶은 뒤 집계 결과를 조건으로 걸러내는 것은 HAVING 절의 역할이다. WHERE는 그룹화 이전 단계라 집계 함수를 조건으로 쓸 수 없다.

선택지별 풀이

① WHERE는 GROUP BY 이전에 개별 행을 거르는 단계라 SUM/COUNT 같은 집계 함수를 조건에 쓸 수 없다.

② GROUP BY는 그룹을 묶는 기준을 정할 뿐 필터링 절이 아니다.

③ 정답. HAVING은 그룹 집계 결과에 대해 조건을 거는 전용 절이다.
④ ORDER BY는 정렬용이며 필터링 절이 아니다.

> **보충 개념** 실행 순서는 FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY 이다. 집계 함수는 GROUP BY로 그룹이 형성된 뒤에야 값이 정해지므로, 그 결과를 거르는 조건은 GROUP BY 다음에 오는 HAVING에만 쓸 수 있다.

41 2과목

다음 정규표현식 함수의 실행 결과로 추출되는 값은?

```
SELECT REGEXP_INSTR('123123123', '312') FROM DUAL;
```

① 1 ② 2 **③ 3** ④ 4

정답 ③

핵심 해설

REGEXP_INSTR는 패턴이 처음 매칭되는 시작 위치를 1부터 반환한다. 문자열 '123123123'을 한 글자씩 보면 1(1) 2(2) 3(3) 1(4) 2(5) 3(6)… 이고, 부분문자열 '312'는 3번째 글자 '3'에서 시작해 '3'(3) '1'(4) '2'(5)로 처음 완성되므로 시작 위치는 3이다.

선택지별 풀이

① 1번째부터 보면 '123'이라 '312'와 다르다.
② 2번째부터 보면 '231'이라 매칭되지 않는다.
③ 정답.
④ 4번째('1')부터 시작하면 '123'이라 더 늦은 매칭이며, 첫 매칭은 3번째다.

> **보충 개념** REGEXP_INSTR(소스, 패턴)은 매칭이 없으면 0, 있으면 매칭이 시작되는 1-base 위치를 돌려준다. 위치 인자를 생략하면 1번째 문자부터 탐색하며 가장 이른 매칭 위치를 반환한다.

42 2과목

다음 [고객] 테이블에서 유효한 이메일 형식(영문자, 숫자, 언더바의 조합 @ 영문자, 숫자, 언더바의 조합.com 형태)을 가진 고객의 수를 계산하는 SQL 문으로 가장 올바른 것은? (Oracle 기준)

고객

고객ID	이메일
100	dev_1@oracle.com
200	master@test
300	admin.user@db.com
400	info@korea.com

① `SELECT COUNT(*) FROM 고객 WHERE REGEXP_LIKE(이메일, '^\w+@\w+\.com$');`
② `SELECT COUNT(*) FROM 고객 WHERE REGEXP_LIKE(이메일, '^\w*@\w*.com$');`
③ `SELECT COUNT(*) FROM 고객 WHERE REGEXP_LIKE(이메일, '.*@.*\.com');`
④ `SELECT COUNT(*) FROM 고객 WHERE REGEXP_LIKE(이메일, '[a-zA-Z0-9]@[a-zA-Z0-9].com');`

정답 ①

핵심 해설

요구 형식은 (영문자·숫자·언더바)+ @ (영문자·숫자·언더바)+ .com 이고 \w가 정확히 [A-Za-z0-9_]에 대응한다. 패턴 '^\w+@\w+\.com$'은 시작/끝을 고정(^ $)하고 점을 \.으로 리터럴 처리하므로, dev_1@oracle.com과 info@korea.com만 매칭되어 2건이 된다. master@test는 .com이 없고, admin.user@db.com은 로컬부에 점이 있어 \w+가 @까지 도달하지 못해 제외된다.

선택지별 풀이

① **정답.**

② 점을 이스케이프하지 않아(.com) '.'이 임의 문자와 매칭되고, \w*는 빈 로컬부도 허용해 형식 검증이 느슨하다.

③ .*가 점·@를 포함한 아무 문자나 허용하고 ^ $ 고정도 없어 admin.user@db.com까지 통과시킨다.

④ 단일 문자 한 글자만(+ 없음) 요구하고 .com의 점도 이스케이프하지 않아 길이가 긴 실제 이메일을 못 잡는다.

> **보충 개념** 정규식에서 점(.)은 임의 한 문자를 뜻하므로 리터럴 마침표는 .으로 써야 한다. ^와 $로 전체를 고정하고 +(1회 이상)로 최소 길이를 강제해야 형식 검증이 정확해진다.

43 2과목

다음 SQL 문의 실행 결과로 가장 적절한 것은?

```
SELECT '''A''''' R1 FROM DUAL;
```

① A　② **'A''**　③ ''A''　④ ''''A''''

정답 ②

핵심 해설

문자열 리터럴 안에서 작은따옴표 두 개('')는 따옴표 한 개를 의미한다. 리터럴은 맨 앞 '로 시작해 맨 뒤 '로 끝나며, 가운데 내용은 ''A''''이다. 이를 두 개씩 묶어 치환하면 '' → ', 다음 A, 그리고 '''' → '' 가 되어 최종 출력은 'A'' 이다.

선택지별 풀이

① 이스케이프된 따옴표가 모두 사라진 결과로, 양쪽 따옴표가 남는다.

② **정답.**

③ 앞쪽 따옴표가 2개가 아니라 1개다(앞 ''→' 하나만 남음).

④ 화면에 보이는 따옴표 수를 그대로 센 오답으로, '' 마다 1개로 줄어든다.

> **보충 개념** Oracle에서 문자열 안에 작은따옴표를 넣으려면 ''로 두 번 적는다. 바깥쪽 따옴표 한 쌍을 벗겨내고 내부의 '' 각각을 '로 환원하면 실제 값을 얻는다. (대안으로 q'[...]' 형식의 인용 연산자도 있다.)

44 2과목

다음 SQL 명령어를 실행했을 때의 결과에 대한 설명으로 적절한 것은?

```
DROP TABLE T1 CASCADE CONSTRAINTS;
```

① T1 테이블만 삭제한다.

② **T1 테이블과 T1 테이블을 참조하는 외부 테이블의 FK 제약조건을 함께 삭제한다.**

③ 자식 테이블의 데이터를 삭제한다.

④ 자식 테이블을 전부 삭제한다.

정답 ②

핵심 해설

CASCADE CONSTRAINTS는 삭제 대상 테이블(T1)의 키를 참조하는 다른 테이블에 걸린 참조 무결성(외래키) 제약조건을 함께 제거한다. T1 자체는 삭제되지만, 자식 테이블이나 그 데이터는 그대로 남는다.

선택지별 풀이

① T1을 참조하는 FK가 존재하면 CASCADE CONSTRAINTS 없이는 오류가 나며, 이 옵션은 FK 제거까지 수행하므로 'T1만 삭제'는 정확한 설명이 아니다.

② **정답.**

③ 자식 테이블의 데이터는 삭제하지 않는다(그것은 DELETE CASCADE의 동작이다).

④ 자식 테이블 자체는 삭제되지 않고, 그 테이블에 걸린 FK 제약조건만 제거된다.

보충 개념 CASCADE CONSTRAINTS(테이블 DROP 옵션)와 ON DELETE CASCADE(데이터 행 연쇄 삭제)는 다르다. 전자는 참조 제약조건을 정리해 테이블을 떨어뜨리는 것이고, 후자는 부모 행 삭제 시 자식 행을 함께 지우는 것이다.

45 2과목

다음은 뷰(View)를 생성하고 이를 조회/수정하는 과정이다. 실행 시 오류(Error)가 발생하는 쿼리는?

```
CREATE VIEW V1 AS
SELECT DEPTNO, SUM(SAL) AS SS
FROM EMP GROUP BY DEPTNO;
```

① `SELECT * FROM V1 WHERE SS > 5000;`

② `SELECT A.DEPTNO, B.DNAME FROM V1 A JOIN DEPT B ON A.DEPTNO = B.DEPTNO;`

③ `SELECT DEPTNO, AVG(SS) FROM V1 GROUP BY DEPTNO;`

④ `INSERT INTO V1 (DEPTNO, SS) VALUES (50, 8000);`

정답 ④

핵심 해설

V1은 GROUP BY와 집계함수(SUM)로 정의된 뷰여서 각 행이 실제 기본 테이블의 단일 행과 1:1로 대응하지 않는다. 이런 그룹/집계 뷰는 갱신 불가능(non-updatable)하므로 INSERT/UPDATE/DELETE가 모두 오류를 낸다. SS는 SUM(SAL)의 결과일 뿐 실제 컬럼이 아니어서 값을 거꾸로 써넣을 수 없다.

선택지별 풀이

① 뷰를 단순 조회하며 별칭 SS로 필터링하는 정상 SELECT다.

② 뷰와 DEPT 테이블을 DEPTNO로 조인하는 정상 조회다.

③ 뷰 결과를 다시 그룹·집계하는 정상 조회다(SELECT는 제약 없음).

④ **정답. 집계·GROUP BY 뷰라 DML(INSERT)이 허용되지 않아 오류가 난다.**

보충 개념 뷰가 갱신 가능하려면 기본적으로 단일 테이블 기반이며 GROUP BY·집계함수·DISTINCT·집합연산 등이 없어야 한다. 이런 요소가 들어간 뷰는 읽기 전용으로만 사용할 수 있다.

46 2과목

다음 트랜잭션 실행 결과로 올바른 것은?

```
create table t1 (no number);
insert into t1 values (1);
insert into t1 values (2);
savepoint sv1;
update t1 set no=4 where no=1;
savepoint sv1;
delete from t1 where no >= 2;
rollback to sv1;
insert into t1 values(3);
commit;
select max(no) from t1;
```

① 1 ② 2 ③ 3 **④ 4**

정답 ④

핵심 해설

같은 이름 sv1을 두 번 선언하면 이전 savepoint는 무효가 되고 가장 최근 선언 지점이 sv1이 된다. 단계별로 추적하면 다음과 같다.

단계	테이블 상태
INSERT 1, 2	{1, 2}
SAVEPOINT sv1 (구, 이후 무시됨)	{1, 2}
UPDATE no=4	{4, 2}
SAVEPOINT sv1 (신, 유효)	{4, 2}
DELETE no>=2	{}
ROLLBACK TO sv1	{4, 2}
INSERT 3 → COMMIT	{4, 2, 3}

최종 행은 {4, 2, 3}이고 MAX(no) = 4이다.

선택지별 풀이

① update로 1은 이미 4가 되었으므로 1은 존재하지 않는다.

② 2는 남아 있지만 최댓값은 아니다.

③ 3도 존재하지만 4보다 작다.

④ **정답.**

> **보충 개념** ROLLBACK TO savepoint는 해당 지점 이후의 변경만 취소하고 트랜잭션 자체는 유지한다. 동일 이름으로 savepoint를 다시 만들면 더 최근 위치로 이동하므로, 여기서는 delete만 되돌려지고 update(no=4)는 보존된다.

47 2과목

다음은 [영업사원] 테이블과 [부서별실적] 테이블이다. 다음 SQL문의 실행 결과로 올바른 것은?

영업사원

사원명	소속부서
SMITH	영업1팀
ALLEN	영업2팀
WARD	영업1팀
MARTIN	영업2팀

부서별실적

실적부서	달성액
영업1팀	3000
영업1팀	5000
영업2팀	1000
영업2팀	2000

```
WITH DeptPerf AS (
SELECT 실적부서, AVG(달성액) AS 평균달성액
FROM 부서별실적
GROUP BY 실적부서
)
SELECT A.사원명
FROM 영업사원 A
JOIN DeptPerf B ON A.소속부서 = B.실적부서
WHERE B.평균달성액 > 3000;
```

① SMITH, WARD

② ALLEN, MARTIN

③ SMITH, ALLEN, WARD, MARTIN

④ 반환되는 행 없음

정답 ①

핵심 해설

WITH 절 DeptPerf는 부서별 평균 달성액을 구한다. 영업1팀 = AVG(3000, 5000) = 4000, 영업2팀 = AVG(1000, 2000) = 1500이다. WHERE 평균달성액 > 3000 조건으로 영업1팀(4000)만 통과하고, 영업1팀 소속 사원은 SMITH와 WARD이다.

선택지별 풀이

① **정답.**
② ALLEN·MARTIN은 영업2팀이고 평균 1500으로 조건(>3000)에 미달한다.
③ 두 팀 모두가 아니라 조건을 만족하는 영업1팀만 남는다.
④ 영업1팀이 조건을 만족하므로 결과가 존재한다.

> **보충 개념** WITH(공통 테이블 표현식)는 인라인 뷰처럼 집계 결과를 임시 이름으로 만들어 메인쿼리에서 재사용한다. 부서로 조인한 뒤 집계값에 대한 필터는 WHERE로 가능하다(여기서는 이미 인라인 뷰로 집계가 끝났기 때문).

48 2과목

다음 함수의 반환 결과로 올바른 것은?

```
SELECT COALESCE(NULL, 1, 2) FROM DUAL;
```

① NULL ② **1** ③ 2 ④ 에러

정답 ②

핵심 해설

COALESCE는 인자 목록을 왼쪽부터 평가해 처음으로 NULL이 아닌 값을 반환한다. 첫 인자 NULL은 건너뛰고 두 번째 인자 1이 NULL이 아니므로 1을 돌려준다.

선택지별 풀이

① 모든 인자가 NULL일 때만 NULL을 반환하는데, 1이 있으므로 아니다.
② **정답.**
③ 1이 먼저 평가되어 채택되므로 2까지 가지 않는다.
④ 정상 동작하는 표준 함수로 에러가 아니다.

> **보충 개념** COALESCE(e1, e2, …)는 NVL의 다중 인자 버전으로, 첫 번째 비-NULL을 반환한다. 모든 인자가 NULL이면 NULL을 반환한다.

49 2과목

다음과 같이 PRIMARY KEY, NOT NULL 제약조건이 설정된 [도서목록] 테이블이 있다. 이 테이블을 복제하여 [추천도서] 테이블을 생성하는 다음 CTAS(Create Table As Select) 구문을 실행했을 때, [추천도서] 테이블에 자동적으로 적용되는 제약조건은?

도서목록

컬럼명	데이터타입	제약조건
도서번호	NUMBER	PRIMARY KEY
도서명	VARCHAR2(100)	NOT NULL

저자명	VARCHAR2(50)

```
CREATE TABLE 추천도서 AS
SELECT * FROM 도서목록 WHERE 도서번호 < 100;
```

① PRIMARY KEY
② **NOT NULL**
③ PRIMARY KEY, NOT NULL
④ 적용되는 제약조건 없음

정답 ②

핵심 해설

CTAS는 컬럼 정의와 데이터, 그리고 명시적으로 선언된 NOT NULL 제약조건만 새 테이블로 복사한다. PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK 같은 제약조건은 승계되지 않는다. 따라서 도서명 컬럼의 NOT NULL만 [추천도서]에 적용된다.

선택지별 풀이

① PRIMARY KEY는 CTAS로 복사되지 않는다(PK에 딸린 NOT NULL 속성도 함께 사라진다).

② **정답.**

③ PK가 복사되지 않으므로 'PRIMARY KEY, NOT NULL' 동시 적용은 틀리다.

④ 명시적 NOT NULL은 복사되므로 '없음'은 아니다.

> **보충 개념** CTAS(CREATE TABLE AS SELECT)는 구조·데이터를 빠르게 복제하지만 인덱스와 대부분의 제약조건은 가져오지 않는다. 예외적으로 NOT NULL 컬럼 제약은 유지된다. PK 등은 별도 ALTER TABLE로 다시 추가해야 한다.

50 2과목

다음 아래의 [T1] 테이블을 생성한 후, 데이터를 삽입하는 SQL 문 중 실행 시 오류(Error)가 발생하지 않고 정상적으로 실행되는 것은?

```
CREATE TABLE T1 (
COL1 NUMBER DEFAULT 9 PRIMARY KEY,
COL2 VARCHAR2(10) NOT NULL
);
```

① `INSERT INTO T1 VALUES (10);`

② `INSERT INTO T1 (COL1, COL2) VALUES (NULL, 'A');`

③ `INSERT INTO T1 (COL2) VALUES ('A');`

④ `INSERT INTO T1 (COL1) VALUES (10);`

정답 ③

핵심 해설

COL1은 DEFAULT 9가 있는 PRIMARY KEY(NOT NULL), COL2는 NOT NULL이다.
③은 COL2에만 'A'를 넣고 COL1은 생략했으므로 COL1에 DEFAULT 9가 채워진다. 두 컬럼 모두 NOT NULL 조건을 만족하므로 정상 삽입된다.

선택지별 풀이

① 컬럼 목록 없이 값 1개만 주면 컬럼 수(2개)와 값 개수가 달라 'not enough values' 오류가 난다.

② COL1은 PK라 NULL을 허용하지 않으므로 NULL 삽입 시 제약 위반 오류가 난다.

③ **정답.**

④ COL2는 NOT NULL인데 생략되어 NULL이 들어가므로 제약 위반 오류가 난다(COL2에는 DEFAULT가 없다).

> **보충 개념** 컬럼을 생략하면 DEFAULT가 지정된 컬럼은 기본값으로, DEFAULT가 없는 NOT NULL 컬럼은 NULL이 되어 제약 위반이 된다. PRIMARY KEY는 NOT NULL + UNIQUE를 함께 강제한다.